

DPP 3

CS & IT
Algorithms

Q1 Consider an array of n elements. If we apply the linear search algorithm to perform a searching operation on the array, and in worst-case and average case scenario, it takes p and q comparisons and the value of $p + q$ is 41.

Then the maximum number of elements can be present in the array is _____?

Q2 What is the time and space complexity of the binary search algorithm?

- (A) $O(n)$, $O(\log n)$
 (B) $O(\log n)$, $O(n)$
 (C) $O(n)$, $O(n)$
 (D) $O(\log n)$, $O(\log n)$

Q3 Consider the following array

Array	5	3	55	82	40	25	15	30
-------	---	---	----	----	----	----	----	----

How many swaps required to sort the above array using bubble sort _____?

Q4 Consider the following array

Array	3	24	15	232	130	780	1024	512
-------	---	----	----	-----	-----	-----	------	-----

What will be 7th element after 3rd pass of Radix sort ?

- (A) 780 (B) 232
 (C) 512 (D) 130

Q5 Consider an array of n elements where almost all the elements are sorted, rather than one or two elements. Now, if we want to sort the given array, then which of the following sorting algorithms will be efficient ?

- (A) Merge sort (B) Quick sort
 (C) Selection sort (D) Insertion sort

Q6 Which of the following options is/are correct ?

(A) Space complexity of Counting sort is $O(k)$, where k is the size of maximum element in the given input array.

(B) In Bucket sort more than one element belong to same bucket then we use efficient Selection sort to sort them efficiently.

(C) Bucket sort worst case time complexity is $O(n^2)$

(D) Bubble sort is a inplace sorting technique.

Q7 Consider an array A

A	3	5	80	90	2	65	1	10	12
---	---	---	----	----	---	----	---	----	----

If we apply selection sort on the above array then what is the output after 4th pass?

- (A) 1, 2, 3, 5, 65, 90, 80, 10, 12
 (B) 1, 2, 3, 5, 90, 65, 80, 10, 12
 (C) 1, 2, 3, 5, 90, 65, 10, 80, 12
 (D) 1, 2, 3, 5, 12, 65, 80, 90, 10

Q8 Consider there are $4n$ sorted subarrays where each subarray size is $n/4$. Now if we merge them into a single sorted array then in the worst case asymptotically Merge algo will take ?

- (A) $\theta(n \log n)$
 (B) $\theta(n^3 \log n)$
 (C) $\theta(n^2 \log n)$
 (D) $\theta(\sqrt{n} \log n)$

Q9 Which of the following recurrence relations represents the Quick Sort worst-case time complexity scenario?

- (A) $T(n) = 2T(n/2) + O(\log n)$



- (B) $T(n) = 2T(n/2) + O(n)$
 (C) $T(n) = T(n-2) + T(1) + O(n)$
 (D) $T(n) = T(n-2) + T(1) + O(\log n)$

Q10 A Min-heap tree containing n^2 elements then time required to find out the maximum element in the given tree is ?

- (A) $\theta(1)$
 (B) $\theta(\log n)$
 (C) $\theta(n^2)$
 (D) $\theta(n^2 \log n)$

Q11 Consider the array elements:

10, 2, 7, 15, 9, 20, 23

Construct a Max heap tree where we insert all the elements one at a time. Then, which one of the following option is represent the correct sequence of Max heap tree?

- (A) 23, 10, 20, 2, 9, 7, 15
 (B) 23, 15, 20, 2, 9, 10, 7
 (C) 23, 15, 20, 9, 2, 10, 7
 (D) 23, 15, 20, 2, 9, 7, 10

Q12 Which of the following statements is/are not false?

- (A) If we decrease an element of a min-heap tree of n elements, then the heapify purpose it may take $O(\log n)$ time.
 (B) Quick sort and Merge sort are both stable algorithms.
 (C) Quick sort, Merge sort, and Heap sort all have the same worst and average case time complexity.
 (D) If we apply Quick sort on a sorted array of n elements, then it takes $O(n^2)$ time.



Answer Key

Q1 27
Q2 D
Q3 13
Q4 C
Q5 D
Q6 A, C, D

Q7 B
Q8 C
Q9 C
Q10 C
Q11 B
Q12 A, D



[Android App](#)



[iOS App](#)



[PW Website](#)

Hints & Solutions

Q1 Text Solution:

In worst case no of comparison = $n = p$

In average case no of comparison

$$= \frac{1+2+3+4+\dots+n}{n}$$

$$= \frac{\frac{n(n+1)}{2}}{n} = \frac{n+1}{2} = q$$

Now,

$$p + q = 41$$

$$n + \frac{(n+1)}{2} = 41$$

$$2n + n + 1 = 82$$

$$3n + 1 = 82$$

$$3n = 81$$

$$n = 27$$

Q2 Text Solution:

Recurrence relation of binary search algorithm.

$$T(n) = T(n/2) + C, n > 1$$

$$= 1, n = 1$$

$$T(n) = T(n/2) + C$$

$$= T(n/2^2) + C + C$$

$$= T(n/2^2) + 2C$$

$$= T(n/2^3) + 3C$$

∴ after k terms

$$= T(n/2^k) + kC$$

$$\text{Let assume } \frac{n}{2^k} = 1$$

$$n = 2^k$$

$$k = \log_2 n, SC = O(\log_2 n)$$

$$T(n) = T(1) + C * \log_2 n$$

$$= O(\log n)$$

Q3 Text Solution:

5, 3, 55, 82, 40, 25, 15, 30

Total elements = 8

Required 7 passes to sort the above array.

	Comparison	Swap
Pass1: 5, 3, 55, 82, 40, 25, 15, 30 3, 5, 55, 82, 40, 25, 15, 30 40, 92, 30, 82 25, 15, 30, 82	7	5
Pass2: 3, 5, 55, 82, 40, 25, 15, 30 40, 25, 25, 25, 25 25, 15, 30	6	4
Pass3: 3, 5, 40, 25, 15, 30, 55, 82 25, 30, 25, 40 15, 30	5	3
Pass4: 3, 5, 25, 15, 30, 40, 55, 82 15, 25	4	1
Pass5: 3, 5, 15, 25, 30, 40, 55, 82	0	1

Array is sorted now, so further passes there will have no swaps total wsaps = $5 + 4 + 3 + 1 = 13$

Q4 Text Solution:

3, 24, 15, 232, 130, 780, 1024, 512

Maximum number 1024 has 4 digits, so make every number of 4 digit and it requires 4 passes to sort the given array through radix sort

0003, 0024, 0015, 0232, 0130, 0780, 1024, 0512

0003	0024	0015	0232	0130	0780	1024	0512
0780	0512	1024					
0130	0232	0003	0024	0015			
0130	0780	0232	0512	0024	1024	0015	
0003	0015	1024	0232				
0003	0512	0024	0130				
0003	0232	0130	0024	1024	0130	0232	0780
0003	0130	0232		0512	0780		
0003	0130	0232	1024	0301	0232	0512	0780

Q5 Text Solution:

Insertion sort best-case time complexity is $O(n)$, when all the array elements are almost sorted.

Q6 Text Solution:

Option a is true because to implement Counting sort we required an extra array



which is size of the maximum element in the given input array.

Option b is false because in Bucket sort we use Insertion sort, which has a best-case time complexity of $O(n)$.

Option c is true. Bucket sort worst-case time complexity is $O(n^2)$.

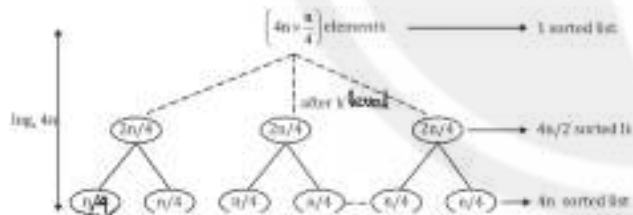
Option d is true in Bubble sort, there is $O(1)$ extra space required to sort the given array. That's why it is an in-place sorting algorithm.

Q7 Text Solution:

3, 5, 80, 90, 2, 65, 1, 10, 12

Pass1: ~~3~~, 5, 80, 90, 2, 65, ~~1~~, 10, 12
 Pass1 o/p → 1, 5, 80, 90, 2, 65, 3, 10, 12
 Pass2: 1, ~~5~~, 80, 90, ~~2~~, 65, 3, 10, 12
 Pass2 o/p → 1, 2, 80, 90, 5, 65, 3, 10, 12
 Pass3: 1, 2, ~~80~~, 90, 5, 65, ~~3~~, 10, 12
 Pass3 o/p → 1, 2, 3, 90, 5, 65, 80, 10, 12
 Pass4: 1, 2, 3, ~~90~~, ~~5~~, 65, 80, 10, 12
 Pass4 o/p → 1, 2, 3, 5, 90, 65, 80, 10, 12

Q8 Text Solution:



Assume after k level we got single sorted array

$$\frac{4n}{2^k} = 1$$

$$2^k = 4n$$

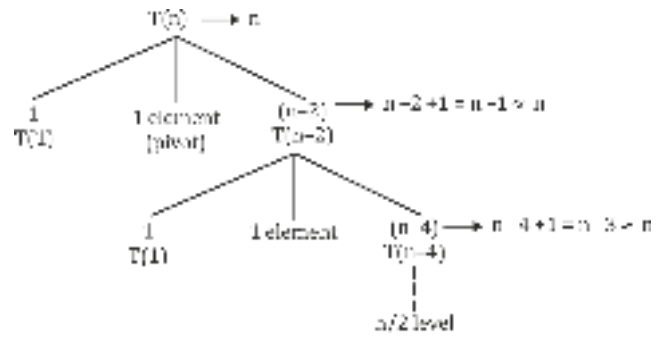
$$k = \log_2 4n$$

each level we moved $(4n \times \frac{n}{4}) = n^2$ elements, so for $\log_2 4n$ level required.

$$\text{Total time} = n^2 \log_2 4n = \theta(n^2 \log n)$$

Q9 Text Solution:

Quick sort worst case happened when partition algo divide the given array into unbalanced way.



$$\text{Total time will be} = n \times \frac{n}{2} = \frac{n^2}{2} = O(n^2)$$

Worst case quick sort general Recurrence Relation

$T(n) = T(C) + T(n-C) + O(n)$, Where C is +ve integer

Q10 Text Solution:

In min heap maximum element present at leaf level.

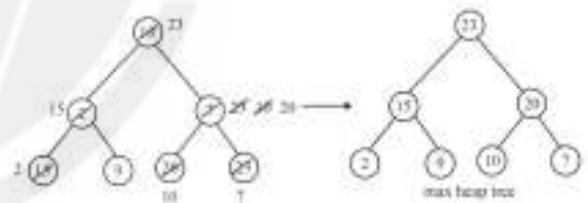
given tree having no of leaf nodes =

$$\left\lceil \frac{n^2}{2} \right\rceil = \frac{n^2}{2}$$

Now Bubble sort 1st pass required to find out maximum element among $\frac{n^2}{2}$ elements which requires asymptotically $\theta(n^2)$ time.

Q11 Text Solution:

Build heap method:



Correct sequence will be 23, 15, 20, 2, 9, 10, 7

Q12 Text Solution:

(a) Decrease key operation on heap tree worst case it take $O(\log n)$ time. (True)

(b) Quick sort is not Stable algorithm. thats why b is false

(c)

Average case

worst case

Merge sort $n \log n$

$n \log n$

Heap sort $n \log n$

$n \log n$

Quick sort $n \log n$

n^2



C is false

(d) If array is already sorted then quick sort worst case happened which take $O(n^2)$ time

true

So, Correct options are a, d.



[Android App](#)



[iOS App](#)



[PW Website](#)

